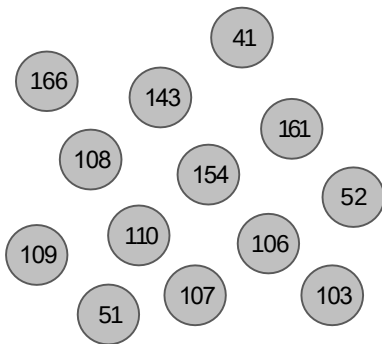


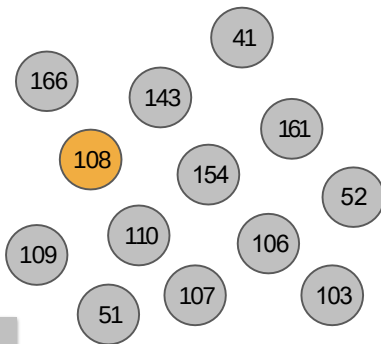
Binary Search Trees: Introduction

Data Structures

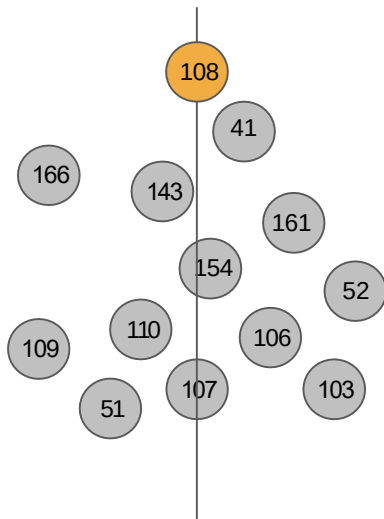
Data Structures and Algorithms

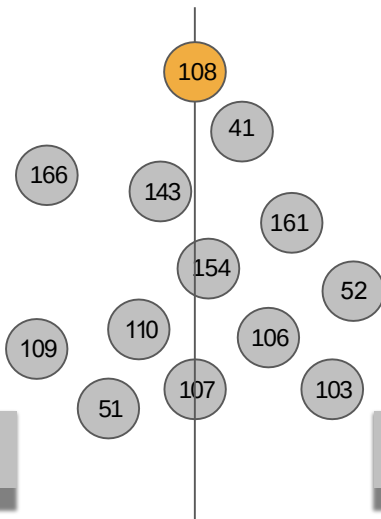
Building a BST





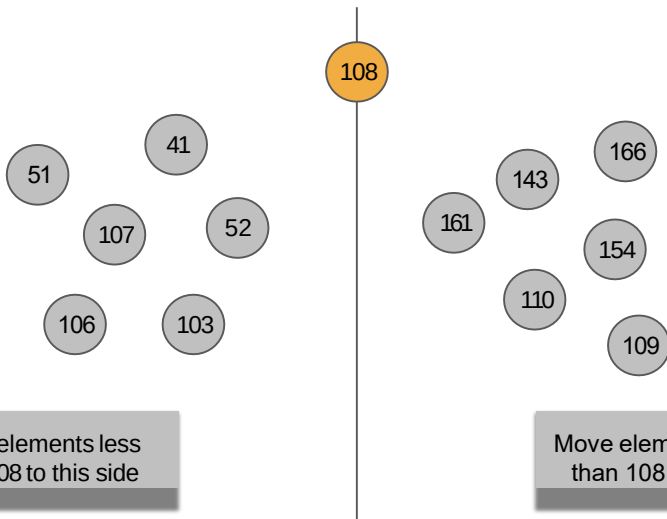
Pick the median element

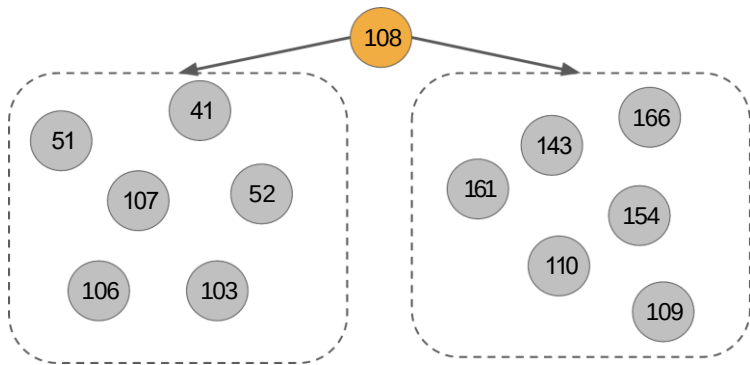


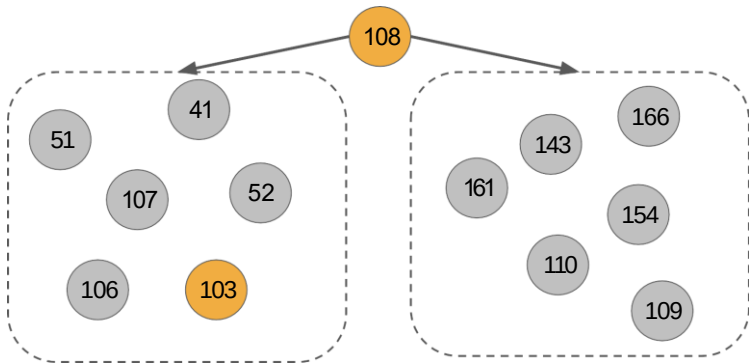


Move elements
less than 108 to this
side

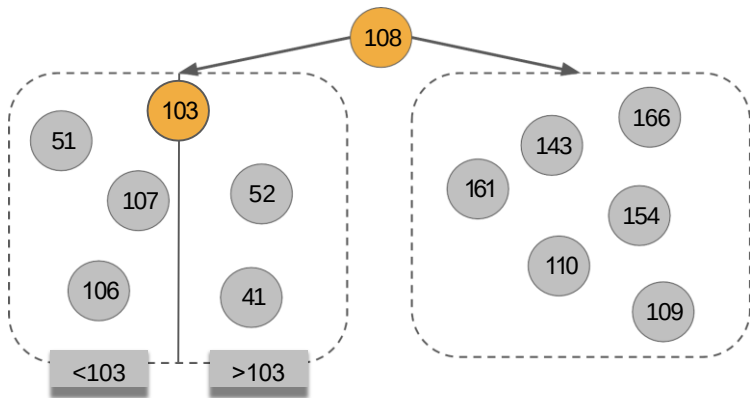
Move elements greater
than 108 to this side

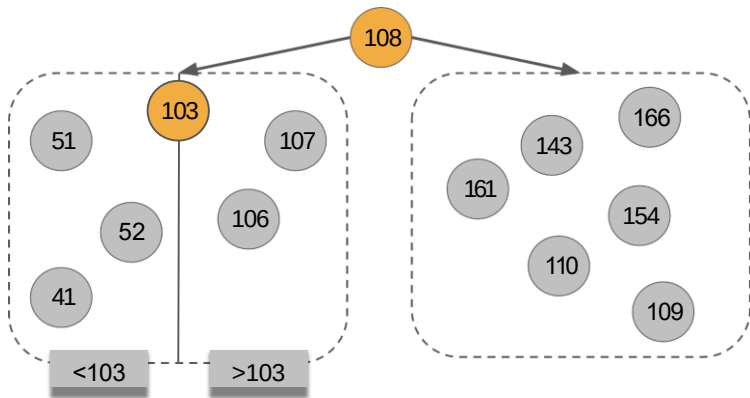


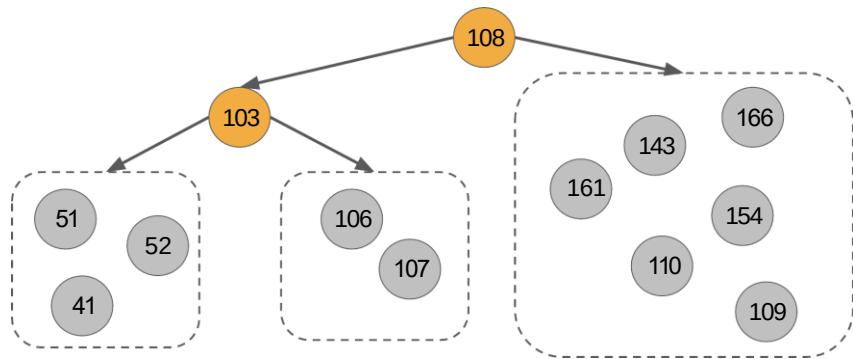


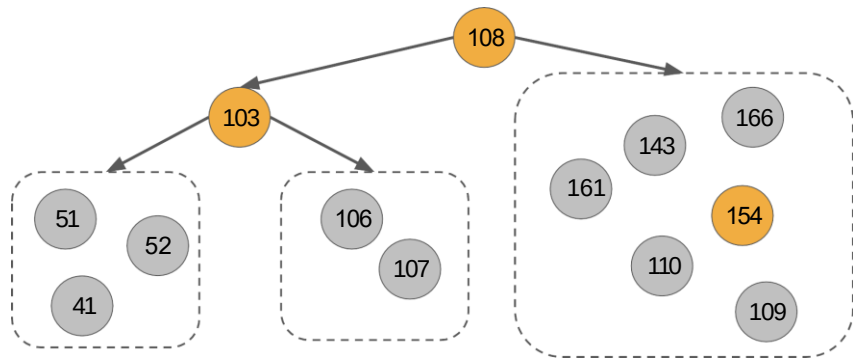


Pick the median element
of the left side

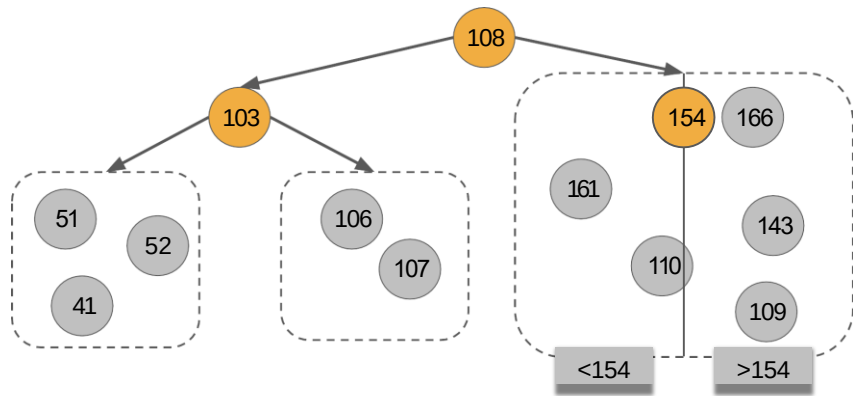


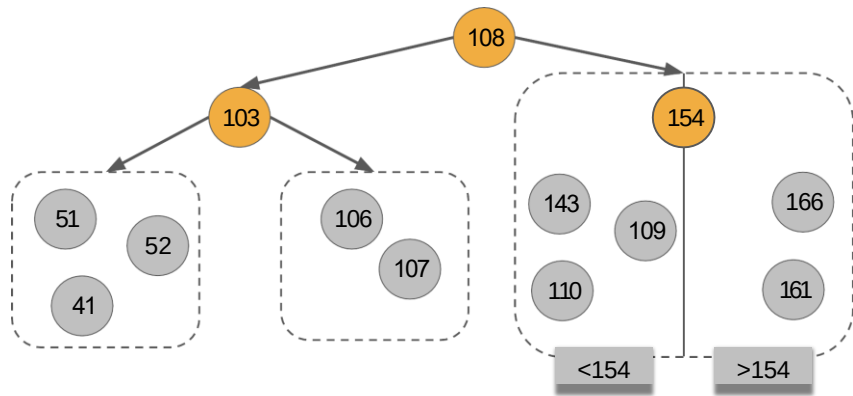


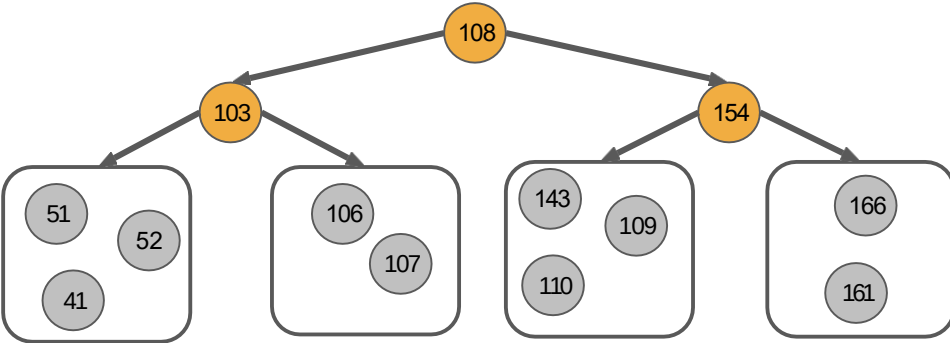


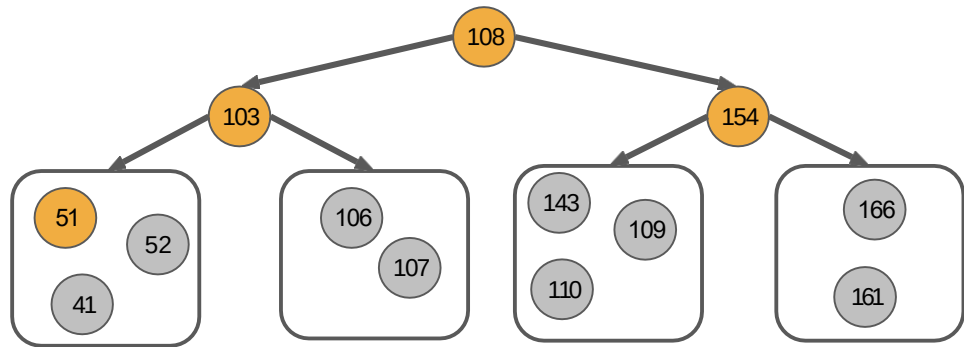


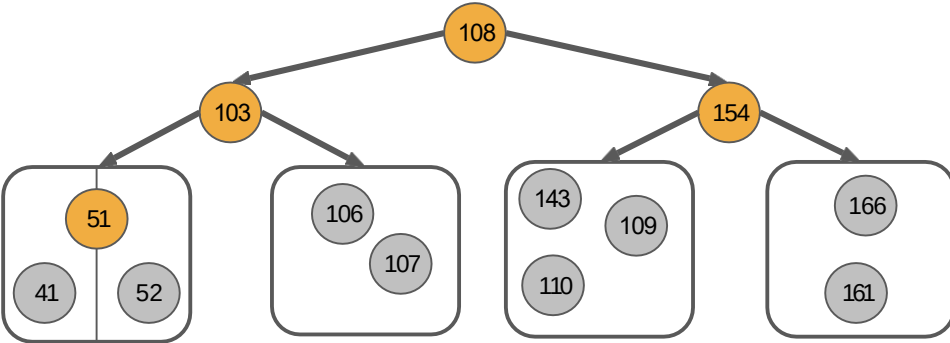
Pick the median element
of the right side

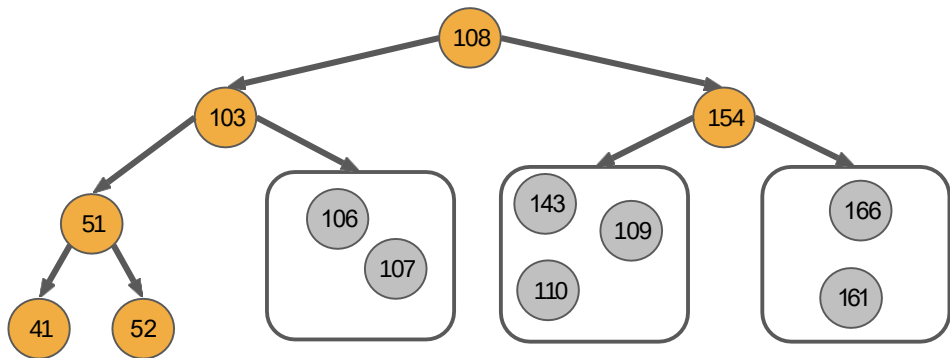


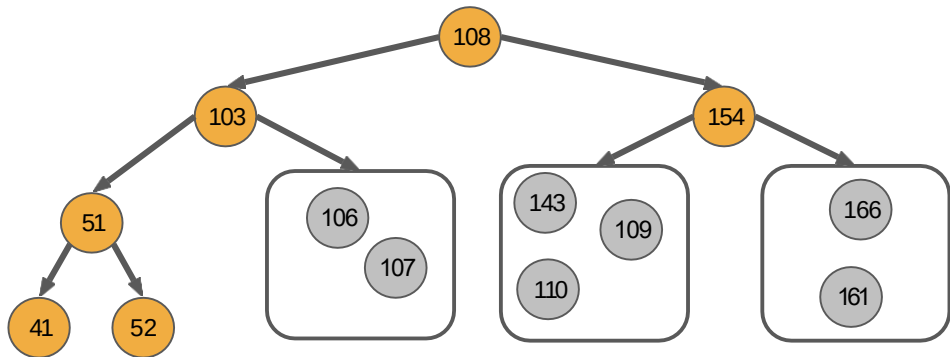




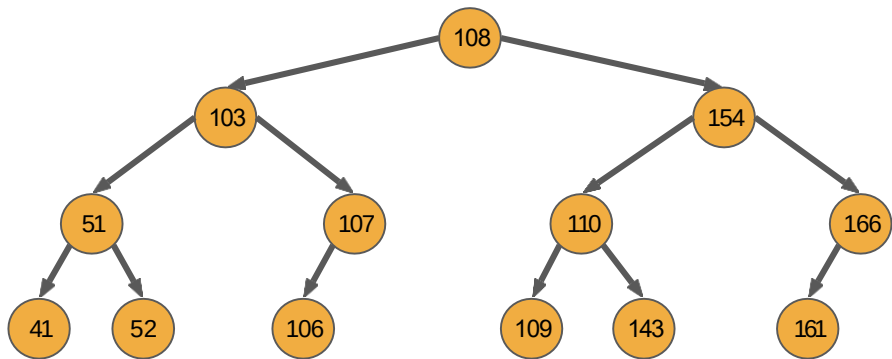


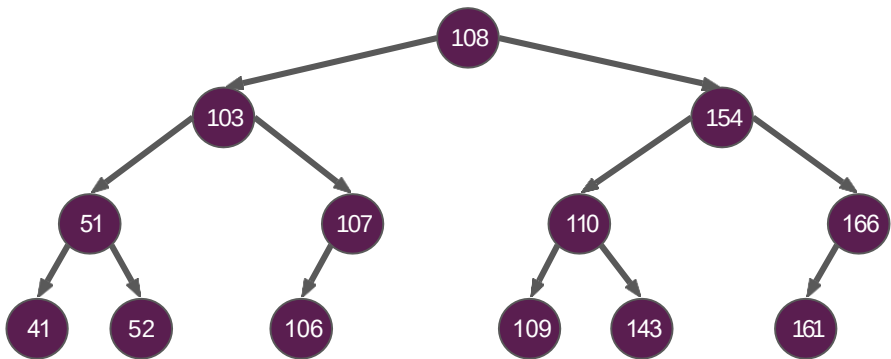




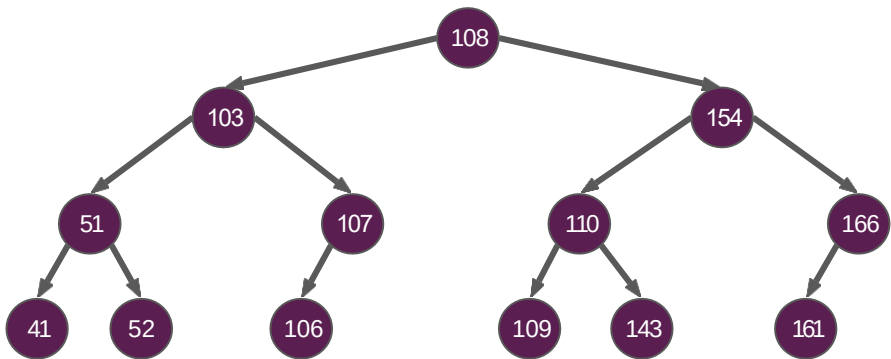


Keep repeating this process
for all the subtrees!



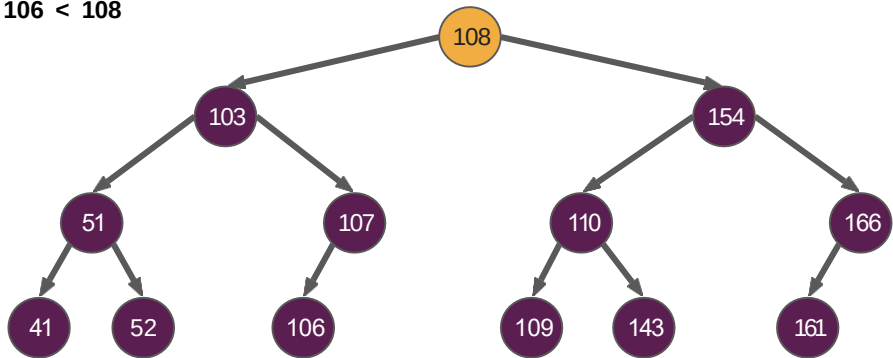


There are 13 nodes in the tree,
but the path to each node is short
($\sim O(\log 13)$)!



How could we check if **106** is in this tree?

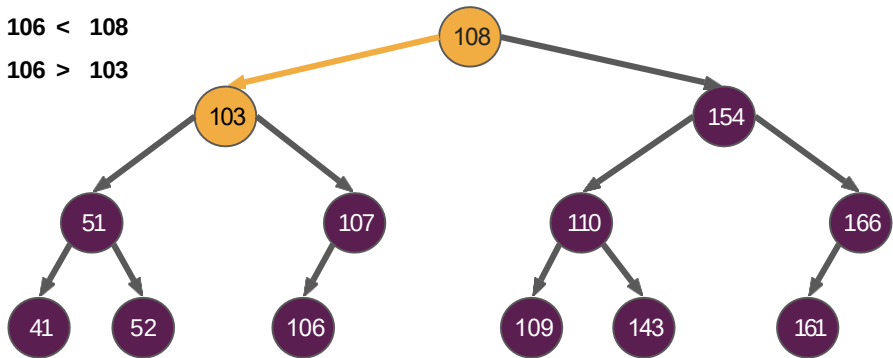
$106 < 108$



How could we check if **106** is in this tree?

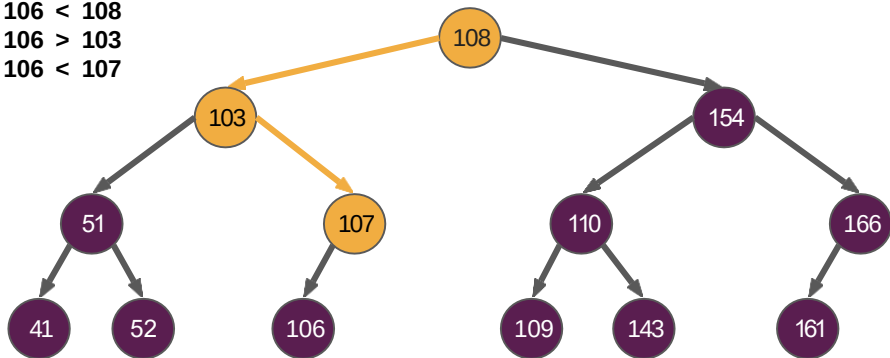
$106 < 108$

$106 > 103$



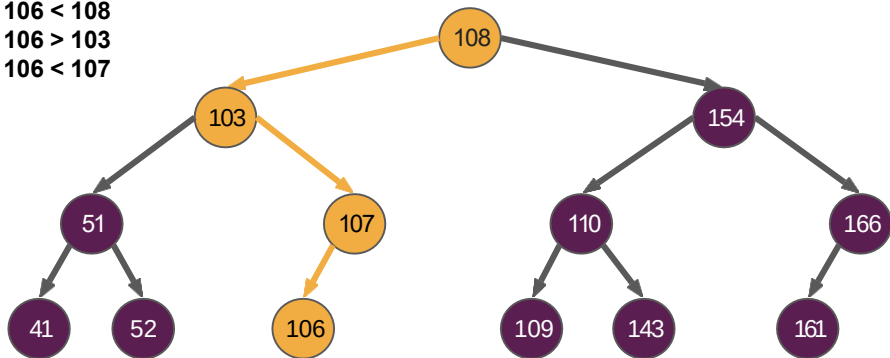
How could we check if **106** is in this tree?

$106 < 108$
 $106 > 103$
 $106 < 107$

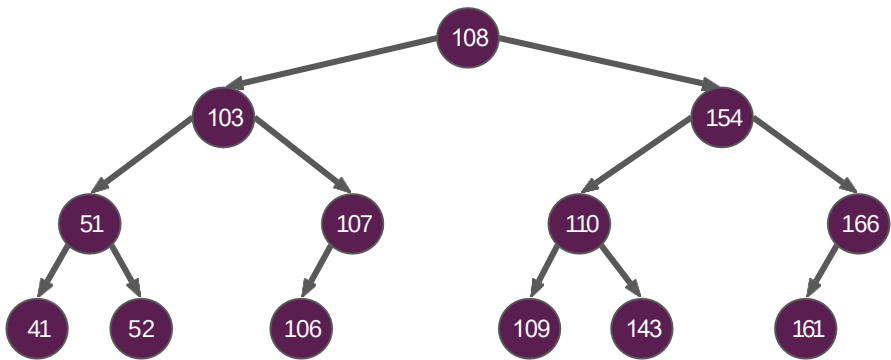


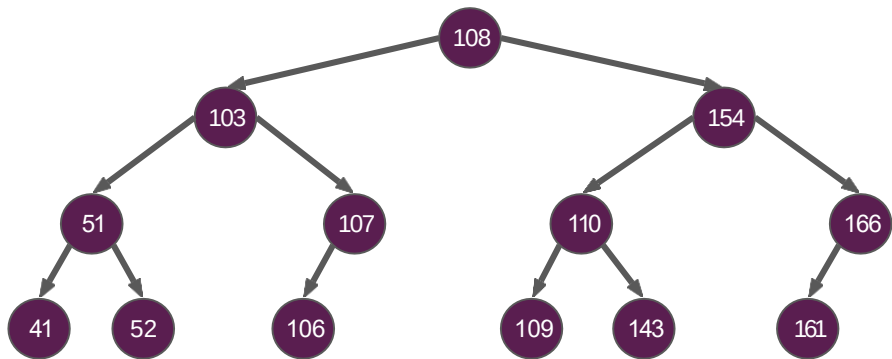
How could we check if **106** is in this tree?

$106 < 108$
 $106 > 103$
 $106 < 107$



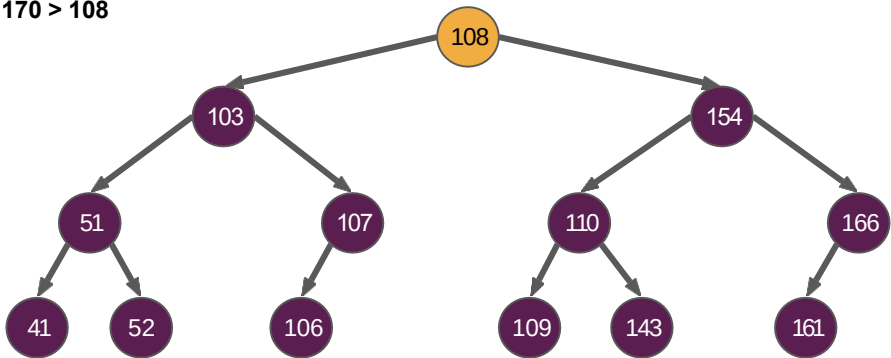
We found **106** so we're done!





How could we check if **170** is in this tree?

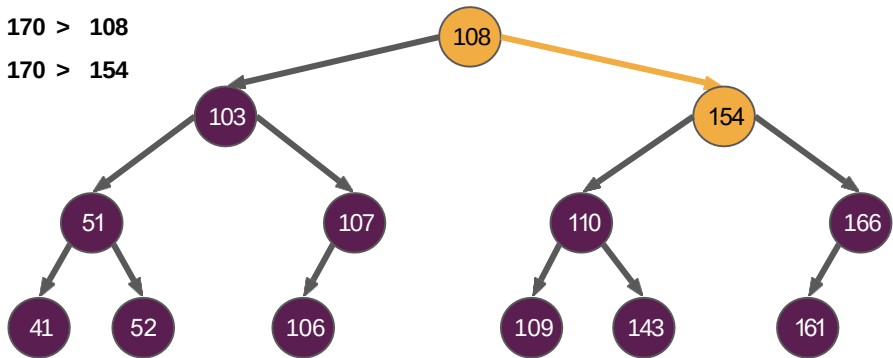
170 > 108



How could we check if **170** is in this tree?

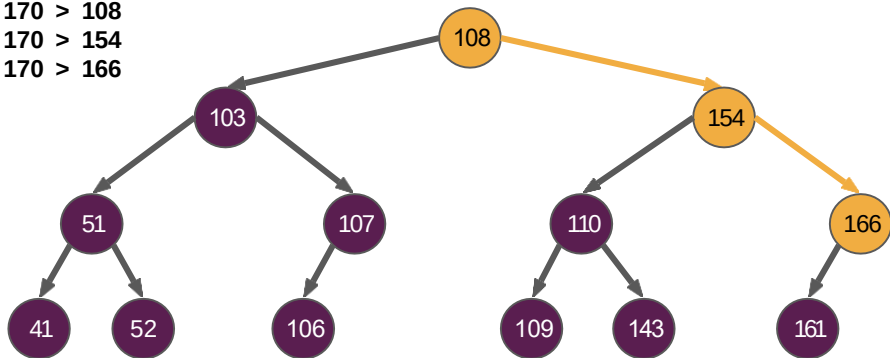
$170 > 108$

$170 > 154$



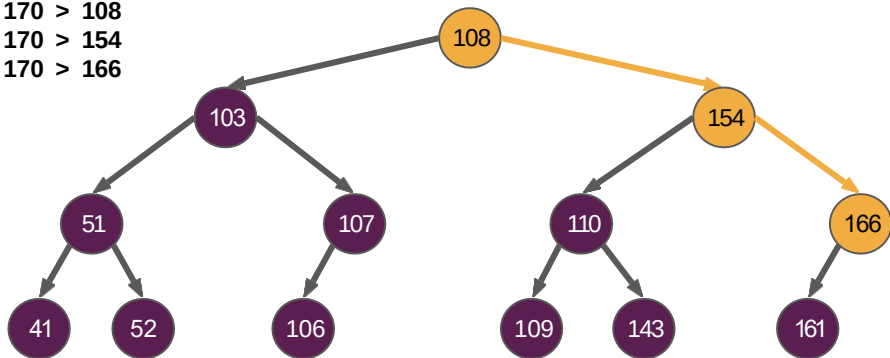
How could we check if **170** is in this tree?

$170 > 108$
 $170 > 154$
 $170 > 166$



How could we check if **170** is in this tree?

170 > 108
170 > 154
170 > 166



Right child is **nullptr** so we're done!

Building a BST

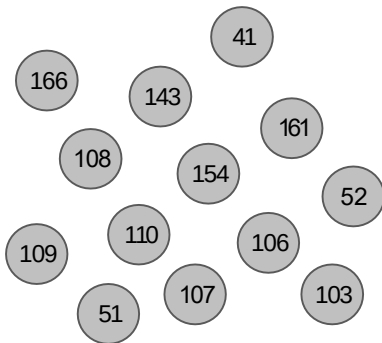
- An **optimal BST** is built by repeatedly choosing the median element as the root node of a given subtree and then separating elements into groups less than and greater than that median.

Building a BST

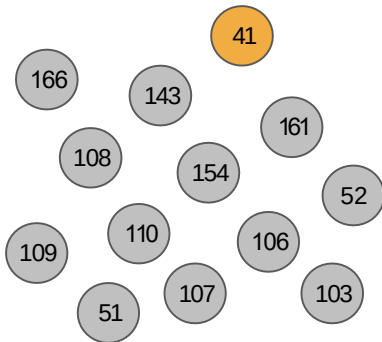
- An **optimal BST** is built by repeatedly choosing the median element as the root node of a given subtree and then separating elements into groups less than and greater than that median.

What does “optimal”
mean?

What if we didn't choose the median?



Let's choose the smallest element instead...



Let's choose the smallest element instead...

41

166

143

161

108

154

52

109

110

106

51

107

103

Let's choose the smallest element instead...

41

166

143

161

108

154

52

109

110

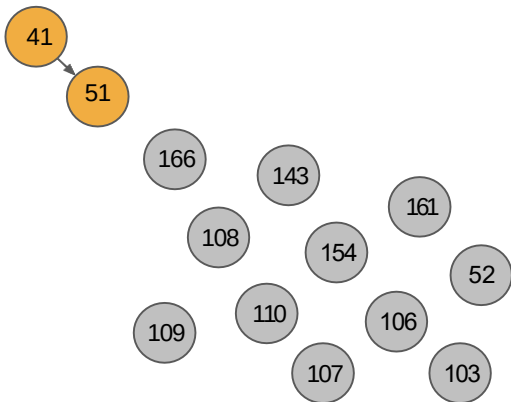
106

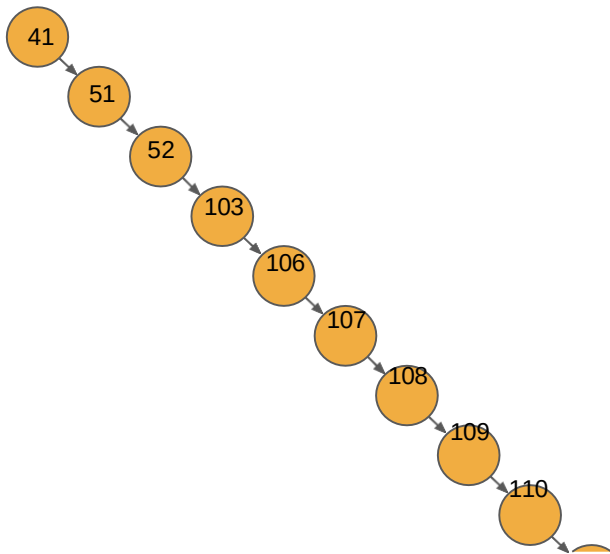
51

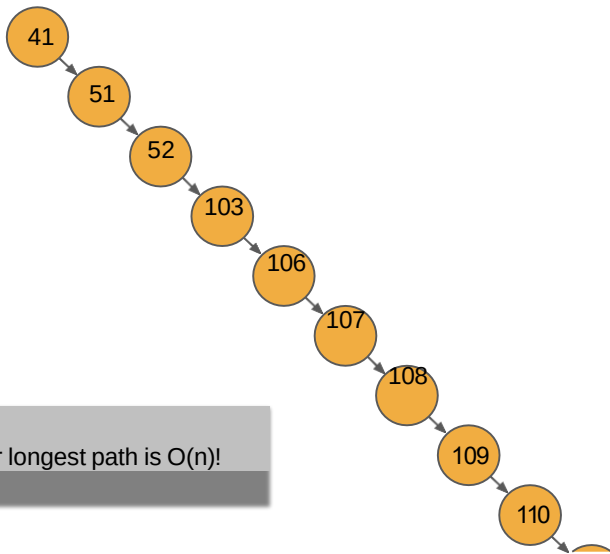
107

103

Let's choose the smallest element instead...







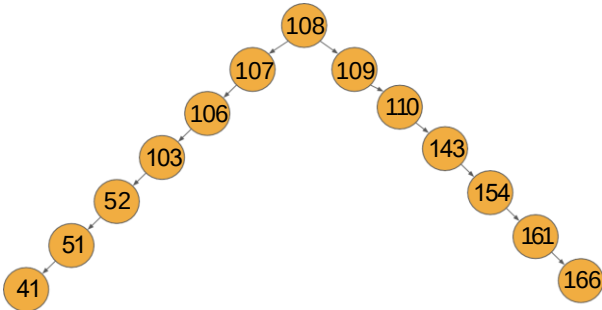
Now our longest path is $O(n)$!

Takeaways

- There are multiple valid BSTs for the same set of data.

Takeaways

- There are multiple valid BSTs for the same set of data.
 - Another example with the previous dataset:



Takeaways

- There are multiple valid BSTs for the same set of data.
- How you construct the tree/the order in which you add the elements to the tree matters!
- A binary search tree is balanced if its height is $O(\log n)$, where n is the number of nodes in the tree (i.e. left/right subtrees don't differ in height by more than 1).
 - Find, insertion, and deletion with balanced BSTs all operate in $O(\log n)$ runtime.